

Faculty of Computer Technology and Cybersecurity
Department of Computer Engineering
ARCHITECTURE AND ORGANIZATION OF COMPUTER SYSTEMS
LABORATORY WORK - 1

Instructor: Dr. Mohsin Farhad, PhD, Professor

Points: 0.90

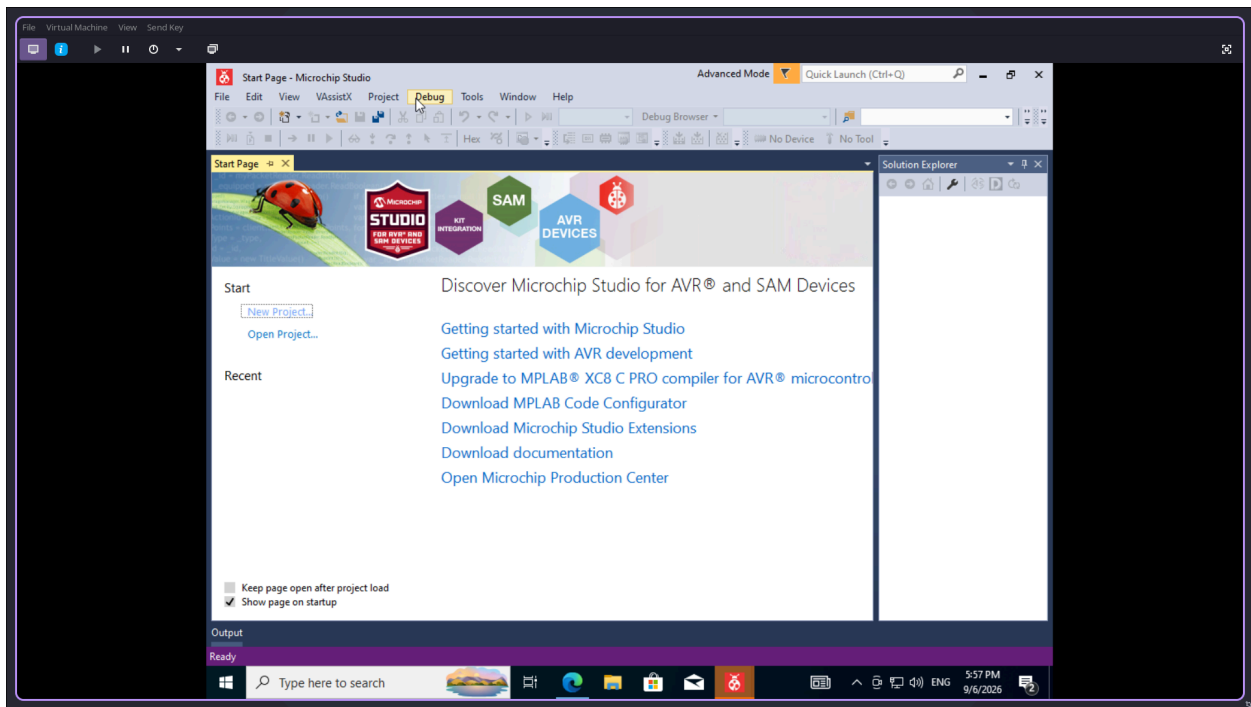
Date: 05.09.2026

Full Name: Daniil Kalts

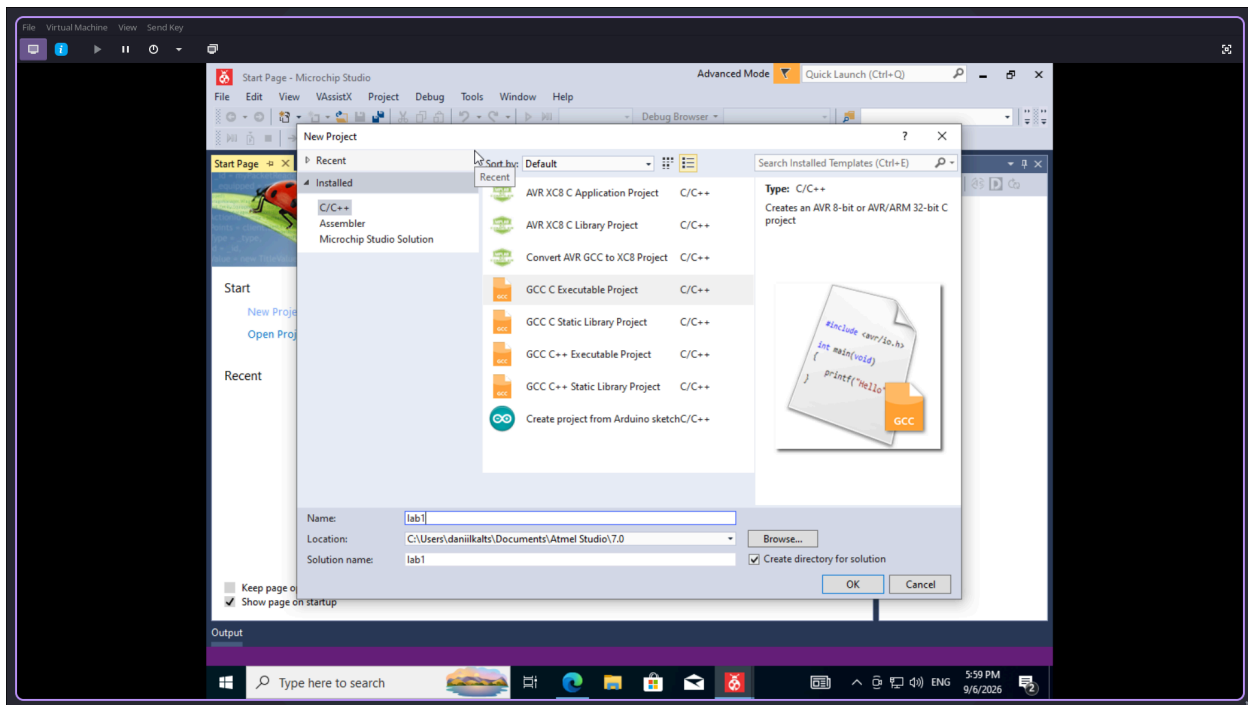
Group: IT2-2404SE

PROGRESS OF WORK

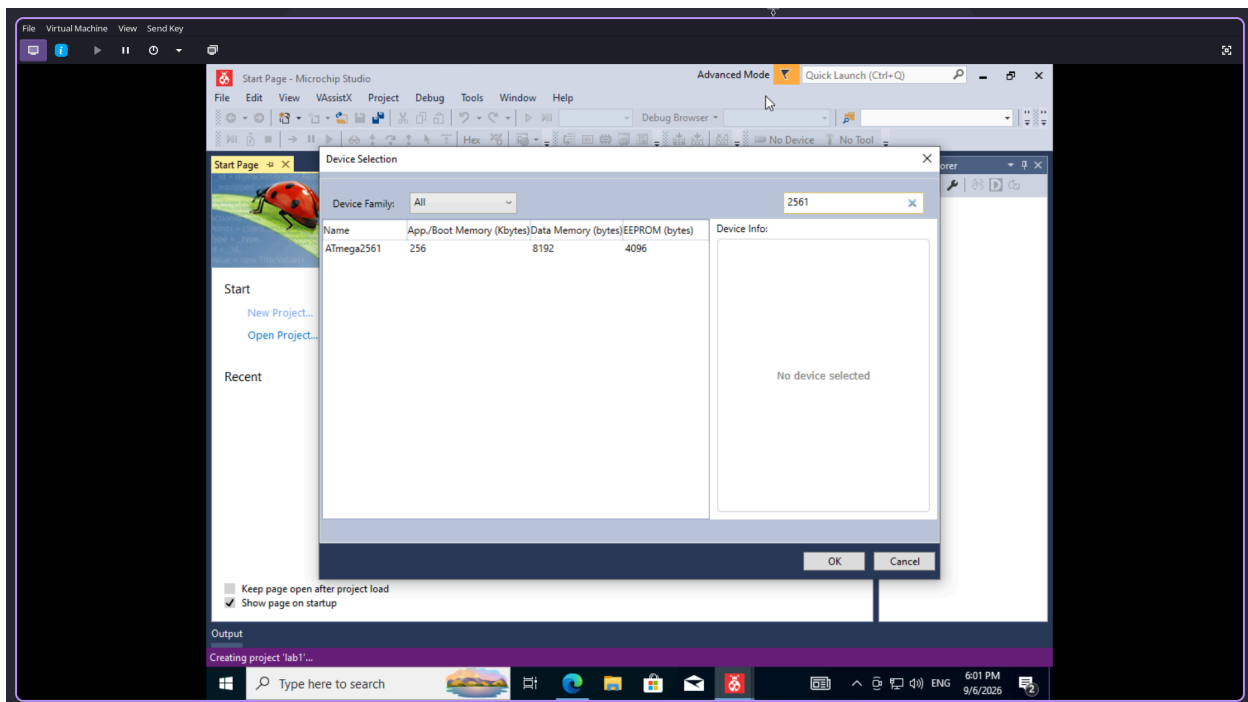
1) Start the Atmel Studio.



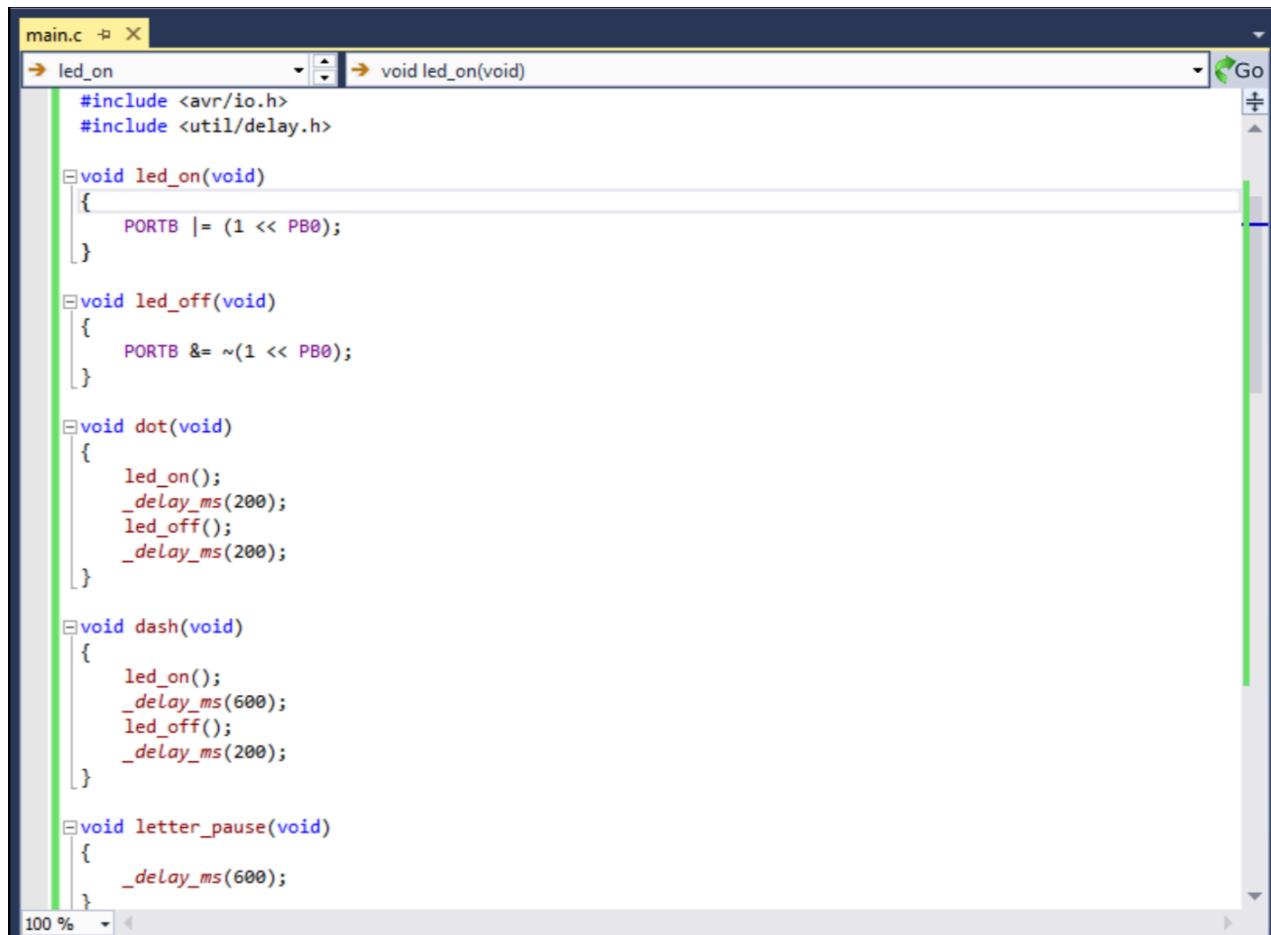
2) Create a new GCC C Executable project.



3) Select the device (in this work — ATmega2561).



4) Write the program implementing Morse code signals (dot(), dashe() and pause() functions) and the main() function that initializes port B and, in an endless loop, outputs the first letters of the first name in Morse code

The image shows a screenshot of a code editor window titled 'main.c'. The editor displays a C program for controlling an LED and generating Morse code signals. The code includes headers for AVR I/O and delays, and defines several functions: led_on(), led_off(), dot(), dash(), and letter_pause(). The led_on() function sets the PB0 pin of PORTB. The led_off() function clears the PB0 pin. The dot() function calls led_on(), delays for 200ms, calls led_off(), and delays for 200ms. The dash() function calls led_on(), delays for 600ms, calls led_off(), and delays for 200ms. The letter_pause() function delays for 600ms. The main function is not fully visible but would typically initialize PORTB and enter an infinite loop to output Morse code for the first letters of a name.

```
main.c
led_on void led_on(void)
#include <avr/io.h>
#include <util/delay.h>

void led_on(void)
{
    PORTB |= (1 << PB0);
}

void led_off(void)
{
    PORTB &= ~(1 << PB0);
}

void dot(void)
{
    led_on();
    _delay_ms(200);
    led_off();
    _delay_ms(200);
}

void dash(void)
{
    led_on();
    _delay_ms(600);
    led_off();
    _delay_ms(200);
}

void letter_pause(void)
{
    _delay_ms(600);
}

100 %
```

```
main.c ✕
→ led_on void led_on(void) Go
void dash(void)
{
    led_on();
    _delay_ms(600);
    led_off();
    _delay_ms(200);
}

void letter_pause(void)
{
    _delay_ms(600);
}

void word_pause(void)
{
    _delay_ms(1400);
}

int main(void)
{
    DDRB |= (1 << PB0);
    led_off();

    while (1)
    {
        // D = -..
        dash();
        dot();
        dot();
        letter_pause();

        // A = .-
        dot();
    }
}
```

100 % Ready Ln 14 Col 2

```
main.c ✕
→ led_on void led_on(void) Go
led_off();

while (1)
{
    // D = -..
    dash();
    dot();
    dot();
    letter_pause();

    // A = .-
    dot();
    dash();
    letter_pause();

    // N = -.
    dash();
    dot();
    letter_pause();

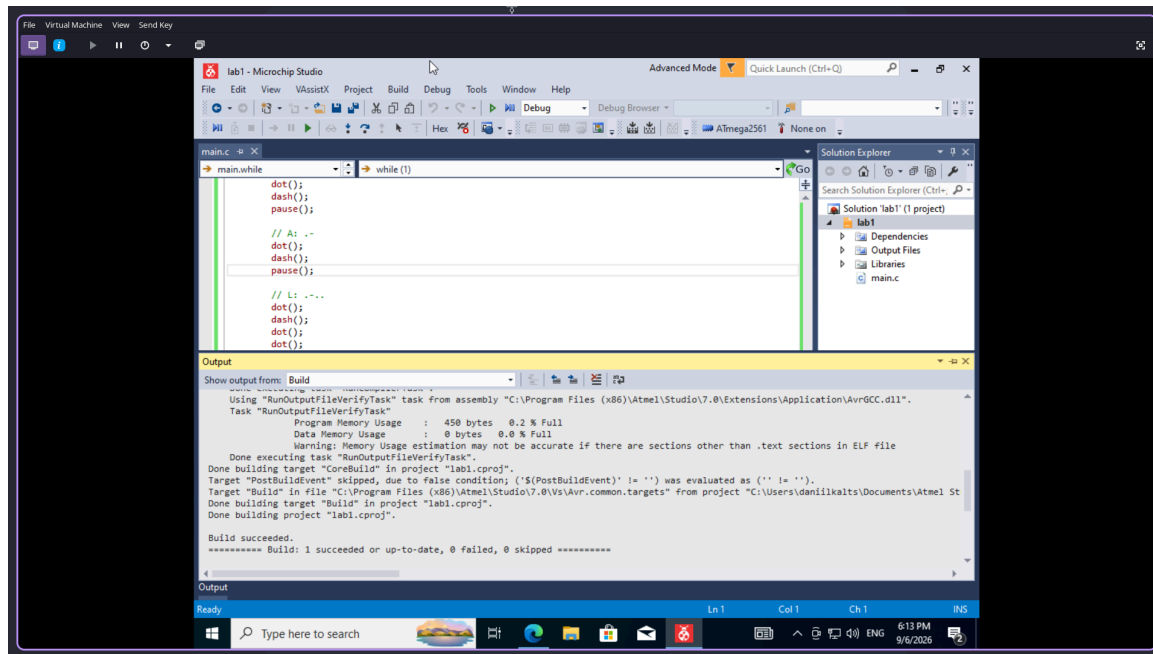
    // I = ..
    dot();
    dot();
    letter_pause();

    // I = ..
    dot();
    dot();
    letter_pause();

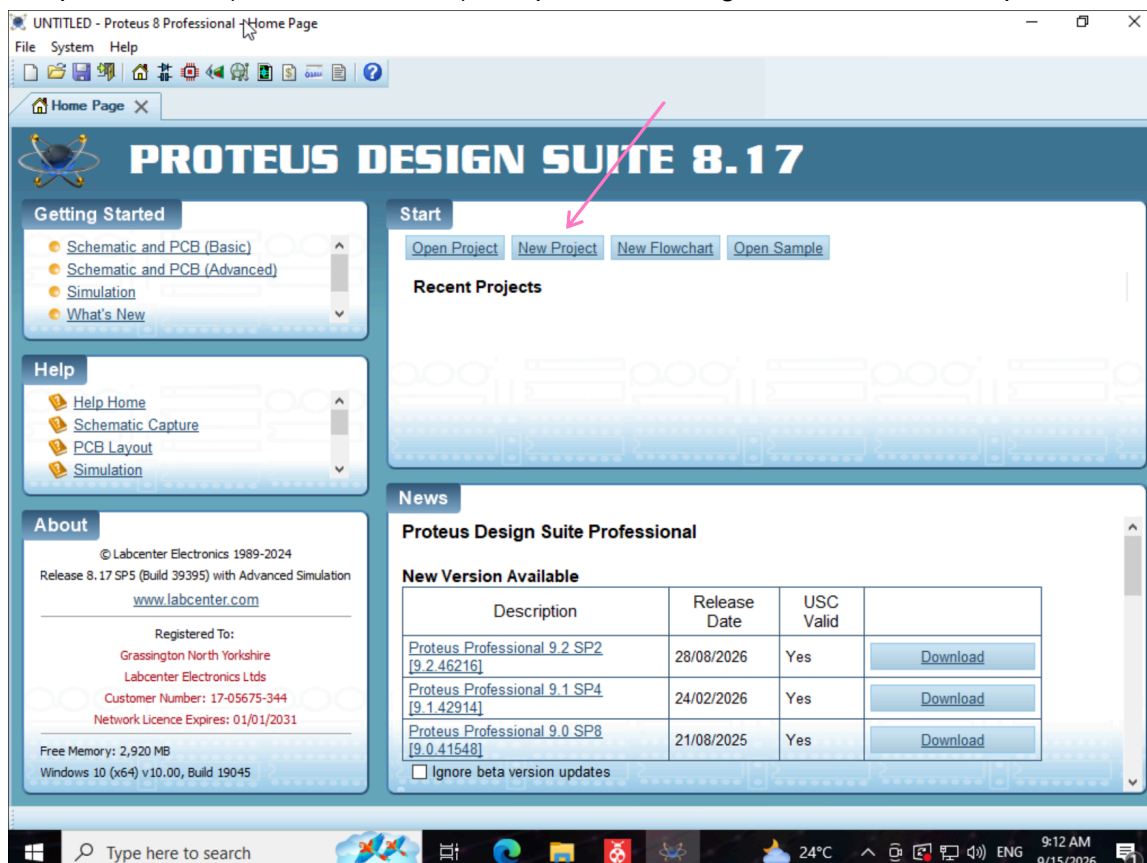
    // L = .-..
    dot();
    dash();
}
```

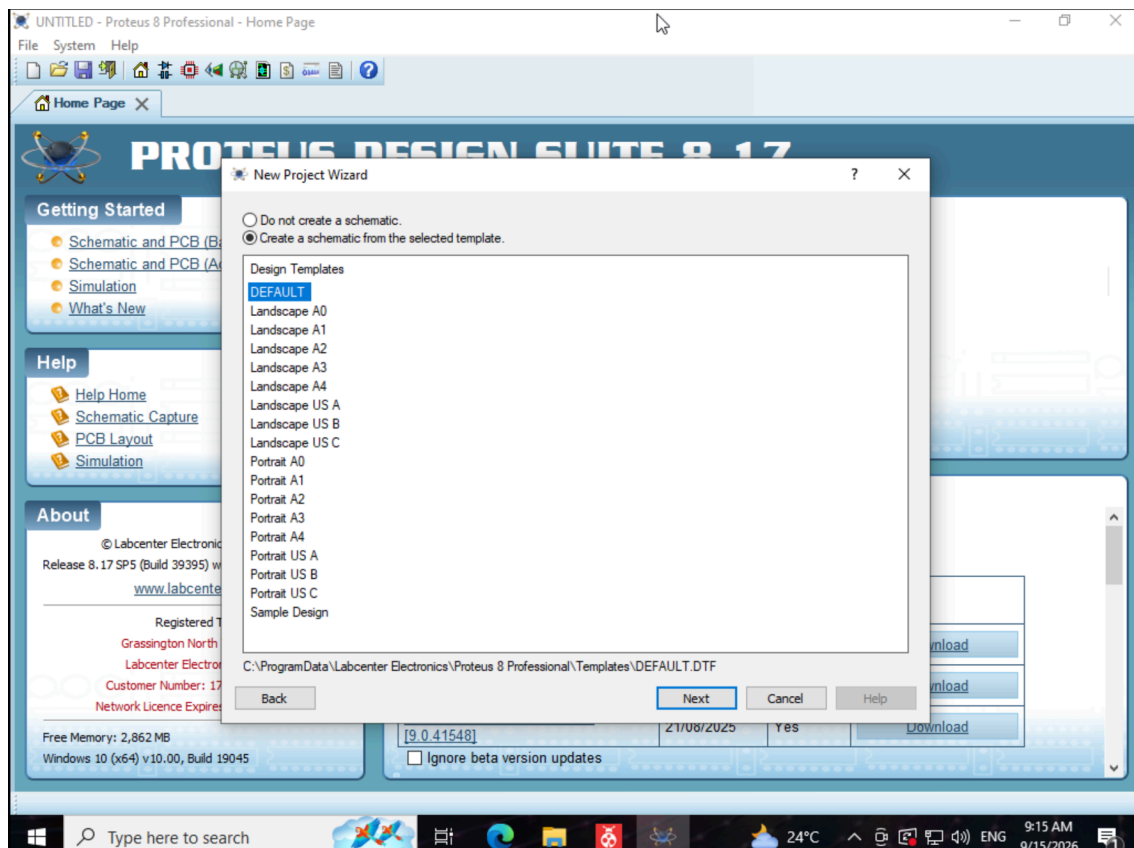
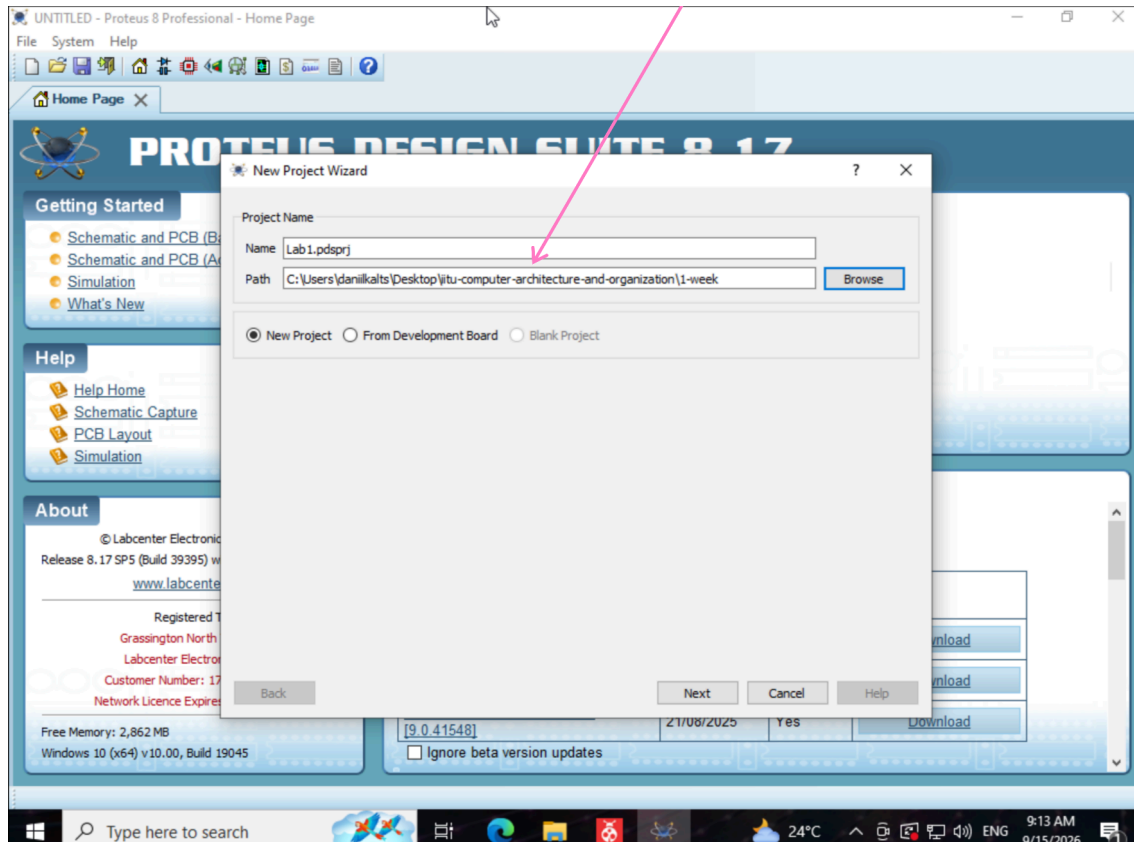
100 % Ready Ln 14 Col 2

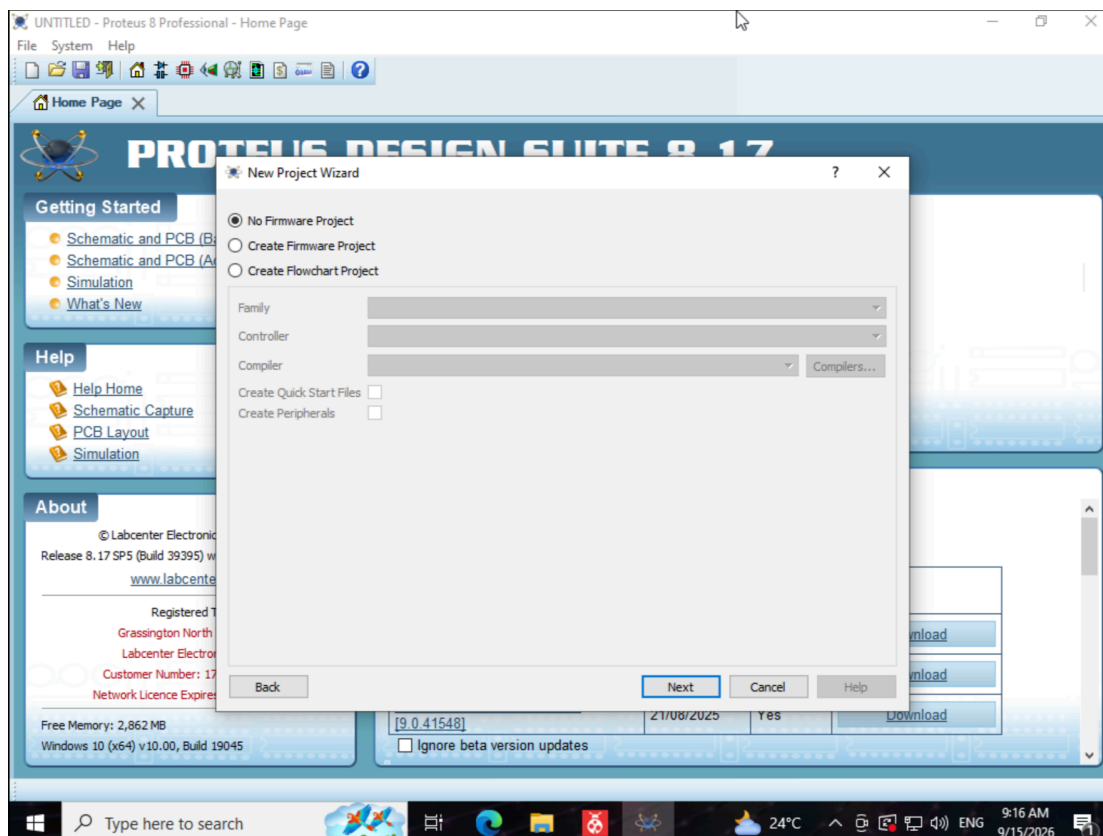
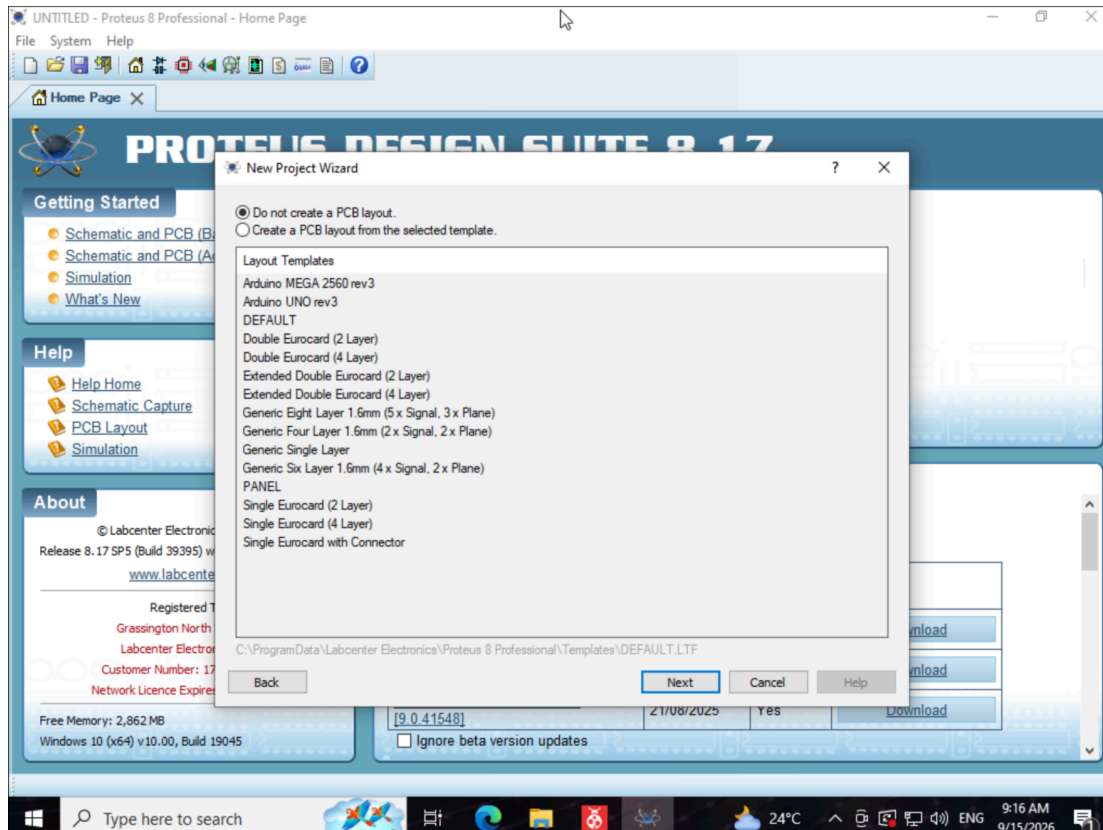
5. Compile the project and obtain the .hex file.



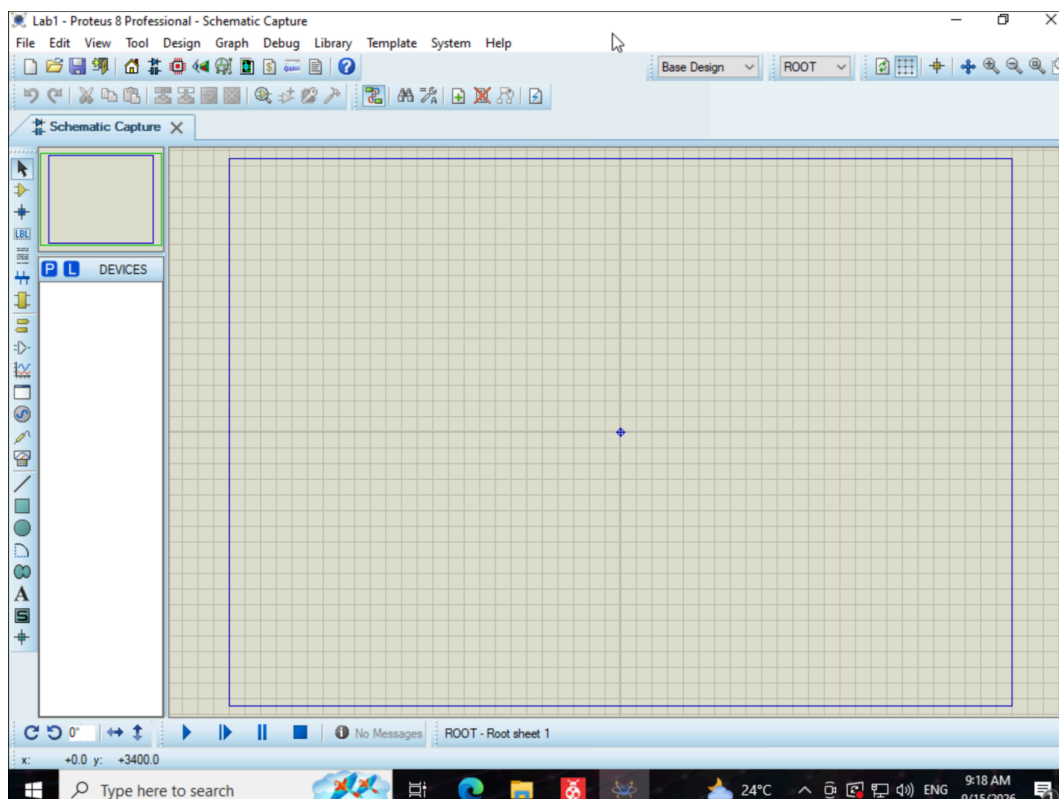
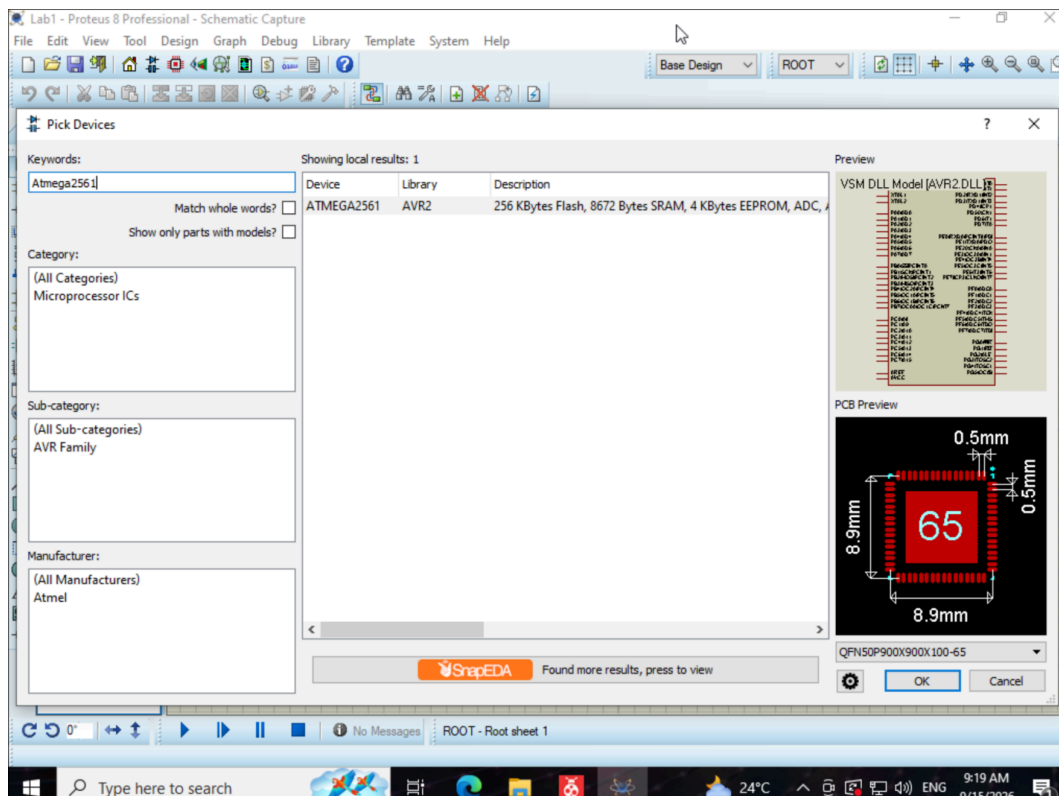
6. Open Proteus (ISIS Professional) and pick the ATmega2561 and LED components.

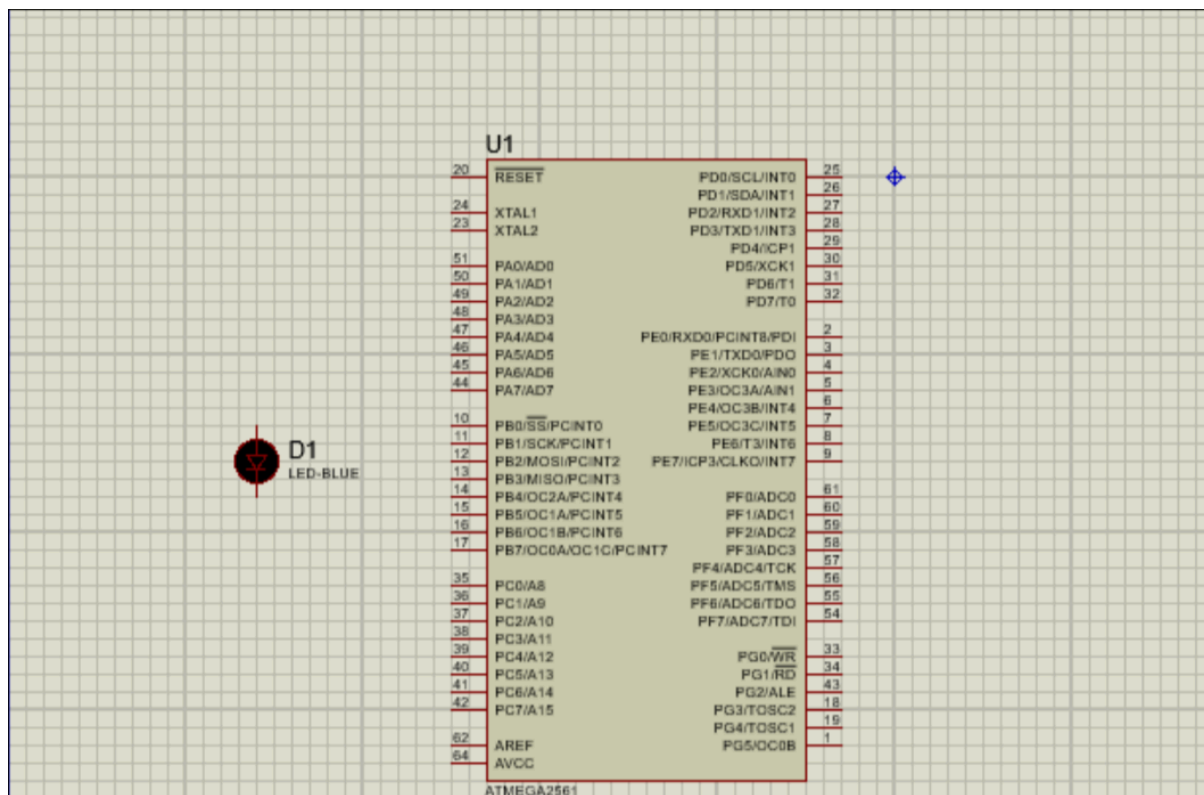
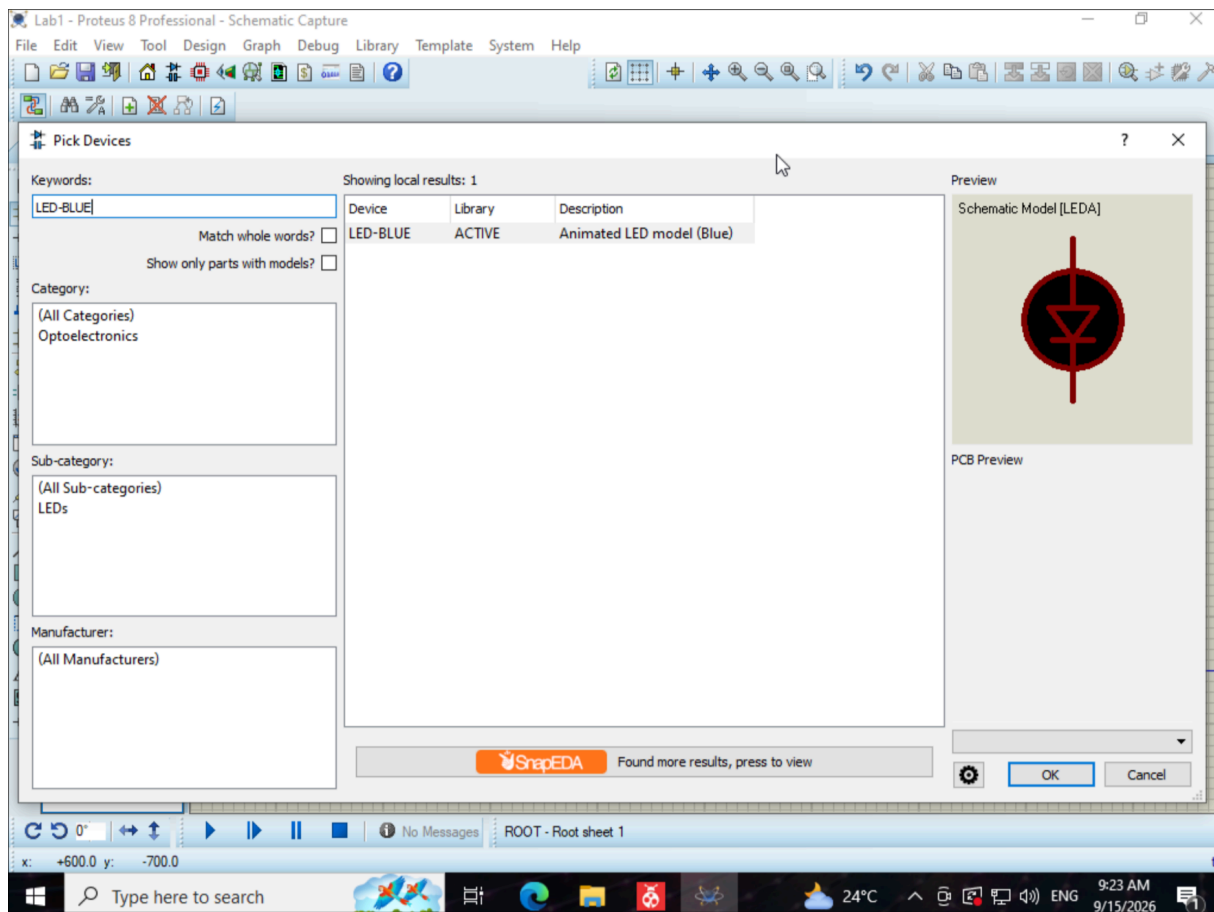




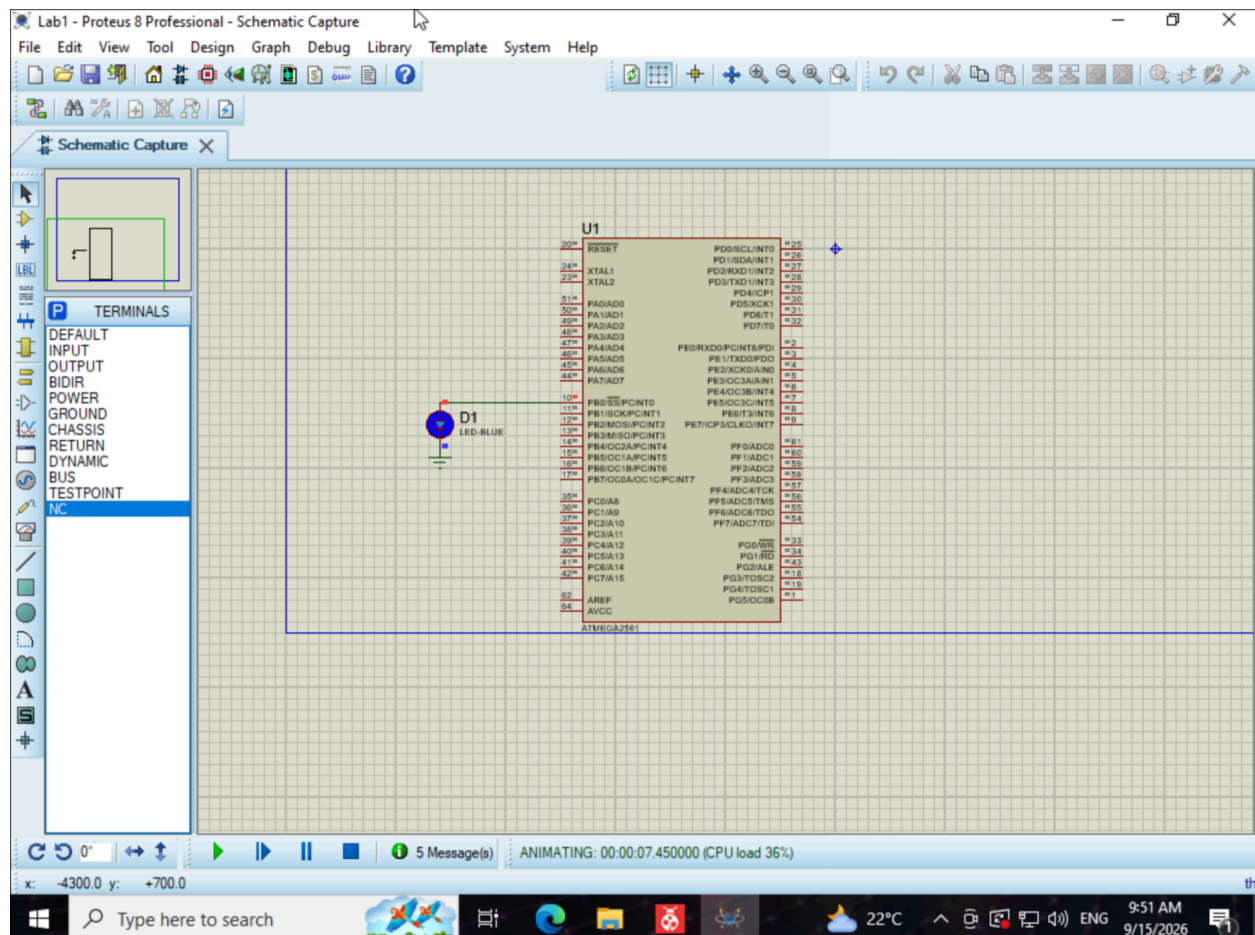


7. Build the circuit: place the microcontroller and the LED on the schematic sheet and connect the pins.





9. Run the simulation and verify that the Morse code (LED blinking) matches the letters of the first and last name.



Conclusion:

In this laboratory work, I programmed the ATmega2561 microcontroller in C and tested the program in Proteus. I configured pin PB0 as an output and connected it to an LED. I created functions for short and long Morse code signals and used them to display the letters of my first name. After compiling the program, I loaded the .hex file into Proteus. The simulation showed the Morse code sequence continuously through the LED.